



LABORATORIO DE PROGRAMACIÓN VIRTUAL
PARA EL DISEÑO LÓGICO DIGITAL
MEDIANTE LÓGICA RECONFIGURABLE

Manual de Usuario

Treviño Hernández Jonathan
Peñarrieta Villa Jesús

Versión 1.0 – 2026

Escuela Superior de Cómputo
Ingeniería en Sistemas Computacionales

Manual de Usuario

Laboratorio de Programación Virtual para el Diseño Lógico Digital
mediante Lógica Reconfigurable

Treviño Hernández Jonathan – Peñarrieta Villa Jesús

2026

Este documento fue elaborado con fines académicos.
Plataforma: Moodle + Virtual Programming Lab (VPL)

Índice general

Prefacio	vii
I Guía del Alumno	1
1 Acceder a la Actividad VPL	3
1.1 Requisitos previos	3
1.2 Ingreso a la plataforma Moodle	3
1.3 Localizar y abrir la actividad VPL	3
2 Entrar al Editor de VPL	5
2.1 Descripción del entorno de edición	5
2.2 Ajustes del editor	5
3 Crear Archivos VHDL	7
3.1 Estructura básica de un archivo VHDL	7
3.2 Crear un nuevo archivo en VPL	7
3.3 Nomenclatura y organización de archivos	7
4 Compilar Archivos VHDL	9
4.1 Proceso de compilación en VPL	9
4.2 Interpretación de errores y advertencias	9
4.3 Buenas prácticas de compilación	9
5 Generación de Código Testbench	11
5.1 ¿Qué es un testbench?	11
5.2 Generación automática en el entorno VPL	11
5.3 Personalización del testbench generado	11
6 Uso de la Simulación en VPL	13
6.1 Iniciar la simulación	13
6.2 Visualización de formas de onda	13
6.3 Depuración mediante simulación	13
7 Uso de la Evaluación Automática	15
7.1 ¿Cómo funciona la evaluación automática?	15
7.2 Enviar el diseño para evaluación	15
7.3 Interpretar la retroalimentación	15
7.4 Reintentos y límite de envíos	15

II	Guía del Profesor	17
8	Creación de Actividad VPL	19
8.1	Precondiciones	19
8.2	Crear una actividad VPL	19
8.2.1	Configuraciones generales	21
8.2.2	Restricciones de entrega	22
8.2.3	Otras configuraciones	23
9	Acciones Permitidas	25
9.1	Tipos de acciones en VPL	25
9.2	Configurar acciones disponibles para el alumno	25
10	Configuración de Casos de Prueba	29
10.1	Archivo <code>vpl_evaluate.cases</code>	29
10.1.1	Sintaxis general de los casos de prueba	29
10.1.2	Sintaxis específica para evaluar código VHDL	31
11	Test de Evaluación por Parte del Profesor	35
A	Glosario de Términos	37
B	Referencia Rápida de Comandos	39
C	Recursos y Referencias	41

Índice de figuras

8.1	Cursos disponibles en Moodle	20
8.2	Cursos donde se desea agregar la actividad VPL	20
8.3	Crear actividad VPL	21
8.4	Configuraciones generales de la actividad VPL	22
8.5	Restricciones de entrega en la actividad VPL	23
8.6	Guardar cambios en la actividad VPL	24
8.7	Visión general de la actividad VPL	24
9.1	Mas configuraciones de la actividad VPL	26
9.2	Opciones de ejecución en la actividad VPL	27
9.3	Permitir opciones de ejecución en la actividad VPL	27

Índice de cuadros

10.1 Parámetros de configuración global	29
10.2 Parámetros por caso de prueba	30

Prefacio

Este manual describe el uso del **Laboratorio de Programación Virtual (VPL)** integrado en *Moodle* para el diseño y verificación de circuitos digitales escritos en VHDL mediante lógica reconfigurable.

El documento se divide en dos partes independientes:

- **Parte I — Guía del Alumno:** acceso, edición, compilación, simulación y evaluación automática de diseños VHDL dentro del entorno VPL.
- **Parte II — Guía del Profesor:** creación y configuración de actividades VPL, definición de casos de prueba, generación de *testbenches* y análisis de reportes de evaluación.

Sobre este documento

Se recomienda leer cada sección de forma secuencial la primera vez y usarlo después como referencia rápida.

Parte I

Guía del Alumno

Acceder a la Actividad VPL

1.1 Requisitos previos

1.2 Ingreso a la plataforma Moodle

1.3 Localizar y abrir la actividad VPL

Entrar al Editor de VPL

2.1 Descripción del entorno de edición

2.2 Ajustes del editor

Crear Archivos VHDL

3.1 Estructura básica de un archivo VHDL

3.2 Crear un nuevo archivo en VPL

3.3 Nomenclatura y organización de archivos

Compilar Archivos VHDL

4.1 Proceso de compilación en VPL

4.2 Interpretación de errores y advertencias

4.3 Buenas prácticas de compilación

Generación de Código Testbench

5.1 ¿Qué es un testbench?

5.2 Generación automática en el entorno VPL

5.3 Personalización del testbench generado

Uso de la Simulación en VPL

6.1 Iniciar la simulación

6.2 Visualización de formas de onda

6.3 Depuración mediante simulación

Uso de la Evaluación Automática

7.1 ¿Cómo funciona la evaluación automática?

7.2 Enviar el diseño para evaluación

7.3 Interpretar la retroalimentación

7.4 Reintentos y límite de envíos

Parte II

Guía del Profesor

Creación de Actividad VPL

8.1 Precondiciones

Antes de crear una actividad VPL, verificar que se cumplan los siguientes requisitos:

- **Cuenta con rol de profesor y permisos de edición:** el usuario debe tener asignado el rol de *Profesor* en *Moodle* con capacidad de editar el curso. Sin este rol, las opciones de configuración de actividades no estarán disponibles.
- **Sesión activa:** el profesor debe haber iniciado sesión en la plataforma *Moodle* antes de realizar cualquier operación.
- **Curso creado:** debe existir al menos un curso en *Moodle* al cual el profesor tenga acceso de edición. La actividad VPL se agrega dentro de un curso existente.
- **Plugin de VPL instalado:** El plugin de VPL debe estar instalado y activo en la plataforma *Moodle*.

Verificación de permisos

Si las opciones de edición no aparecen en el curso, contactar al administrador de la plataforma para revisar el rol asignado a la cuenta.

8.2 Crear una actividad VPL

Para crear una actividad VPL en un curso de *Moodle*, seguir estos pasos:

Paso 1

Ingresa al curso donde se desea agregar la actividad VPL.

CAPÍTULO 8. CREACIÓN DE ACTIVIDAD VPL

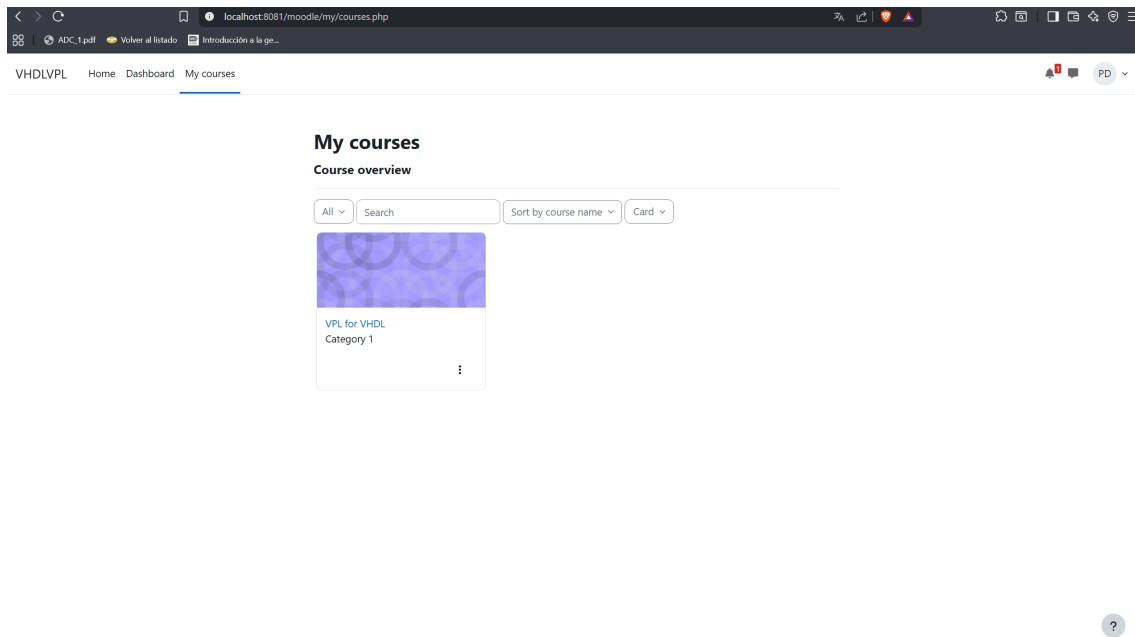


Figura 8.1: Cursos disponibles en Moodle

Paso 2

Activar el modo de edición del curso haciendo clic en el botón **Activar edición**.

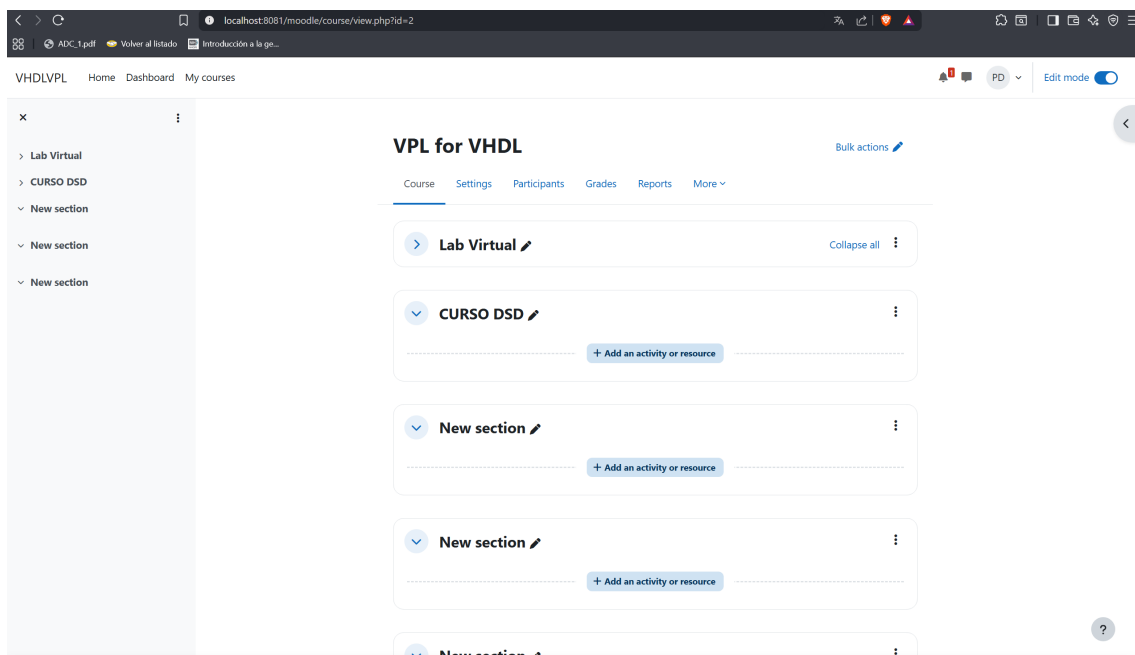


Figura 8.2: Cursos donde se desea agregar la actividad VPL

Paso 3

En la sección del curso donde se desea colocar la actividad, hacer clic en **Añadir una actividad o recurso**. En la ventana emergente, seleccionar **Virtual Programming Lab (VPL)**.

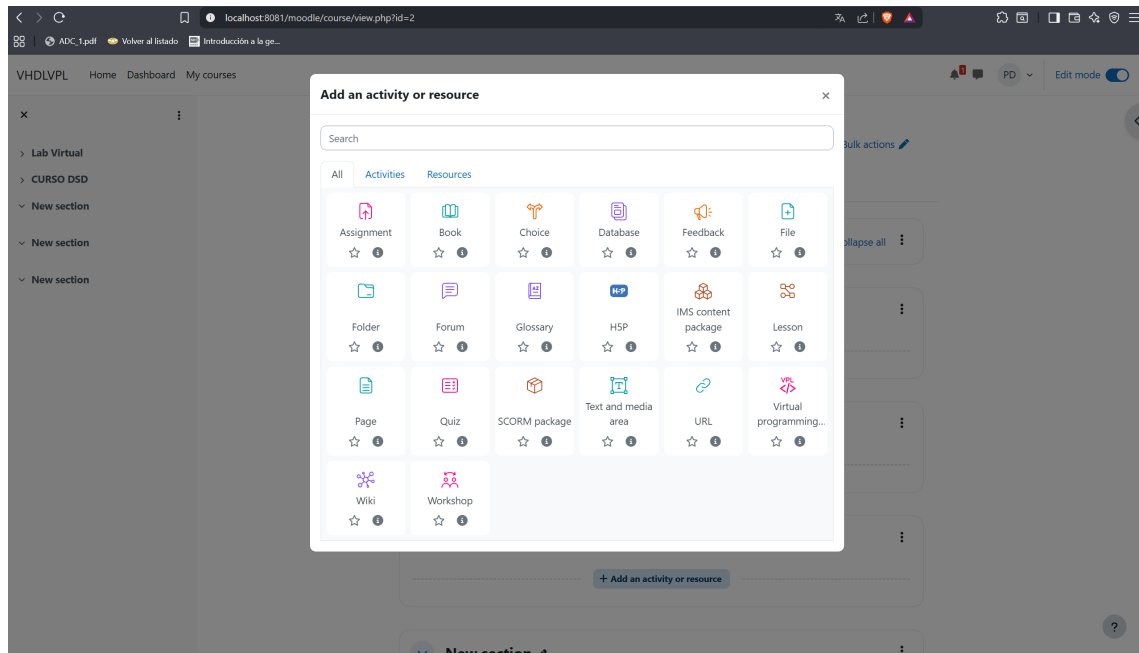


Figura 8.3: Crear actividad VPL

Una vez creada la actividad VPL, se nos muestra una vista para la configuración principal de la actividad, donde se pueden establecer diferentes configuraciones, de las cuales mencionaremos las mas relevantes para el correcto funcionamiento de la actividad, sin embargo, se recomienda revisar cada una de las opciones para personalizar la actividad a las necesidades del curso.

8.2.1 Configuraciones generales

Incluye el nombre de la actividad (obligatorio), una descripción corta y una descripción detallada donde podemos añadir mas recursos (texto, imagenes, videos, etc.).

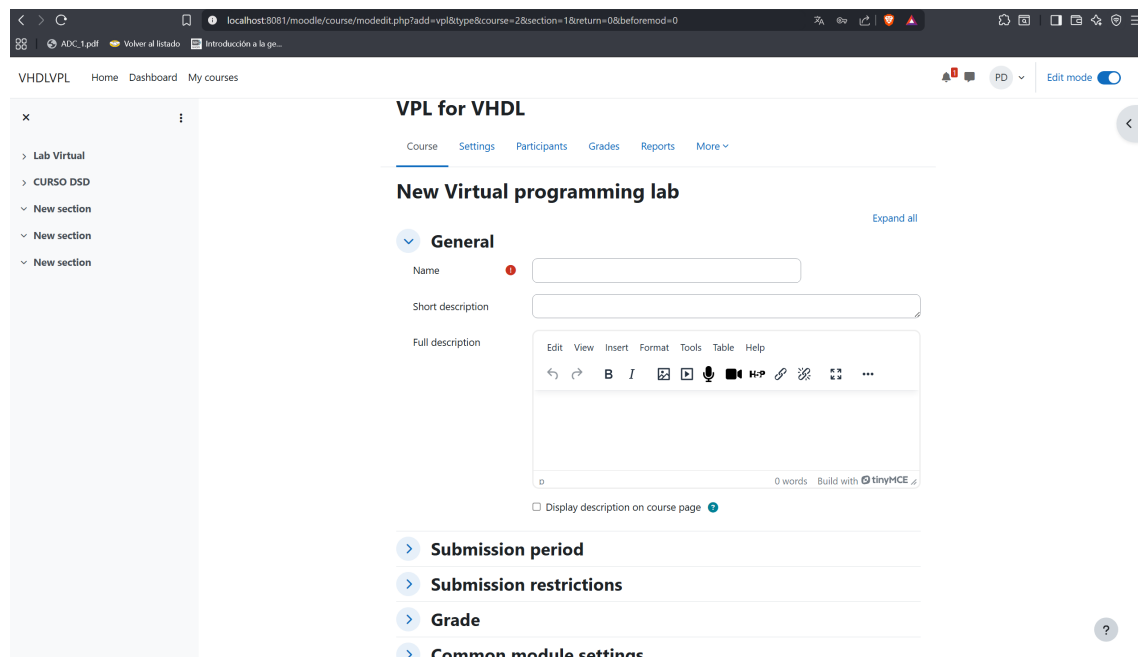


Figura 8.4: Configuraciones generales de la actividad VPL

8.2.2 Restricciones de entrega

En esta sección se pueden establecer configuraciones como el número máximo de archivos que el alumno puede entregar, el tipo de trabajo (individual o grupal), fecha de entrega, entre otras opciones que pueden ser útiles para controlar la entrega de los trabajos por parte de los alumnos. Aquí lo más importante es configurar el número máximo de archivos que el alumno puede entregar, por default se establecieron 2 archivos máximos, considerando un archivo vhdl fuente y su archivo testbench generado, sin embargo, dependiendo de la actividad se pueden establecer más archivos máximos para que el alumno pueda entregar archivos adicionales (por ejemplo, un archivo de texto con una explicación del diseño o un diagrama del circuito).

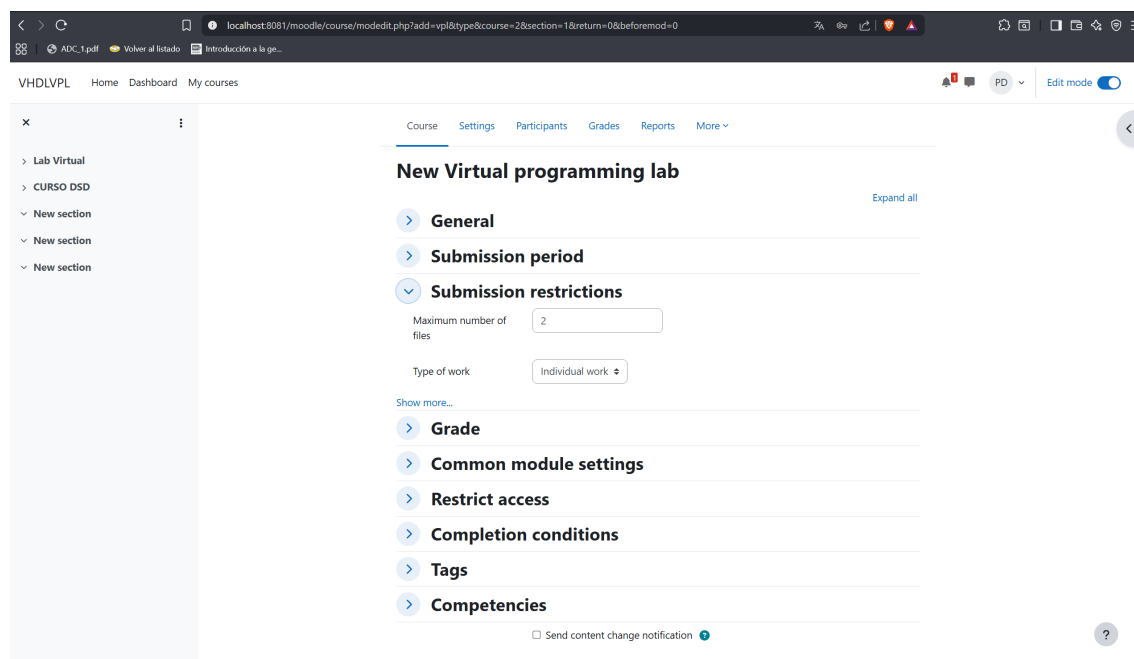


Figura 8.5: Restricciones de entrega en la actividad VPL

8.2.3 Otras configuraciones

EL VPL es un plugin muy completo que ofrece muchas opciones de configuración, por lo que se recomienda revisar cada una de las secciones para personalizar la actividad a las necesidades del curso, sin embargo, se recomienda revisar la documentación propia de VPL for Moodle para saber más acerca de las otras configuraciones.

Una vez configurada la actividad, se debe guardar los cambios para que la actividad quede disponible para los alumnos.

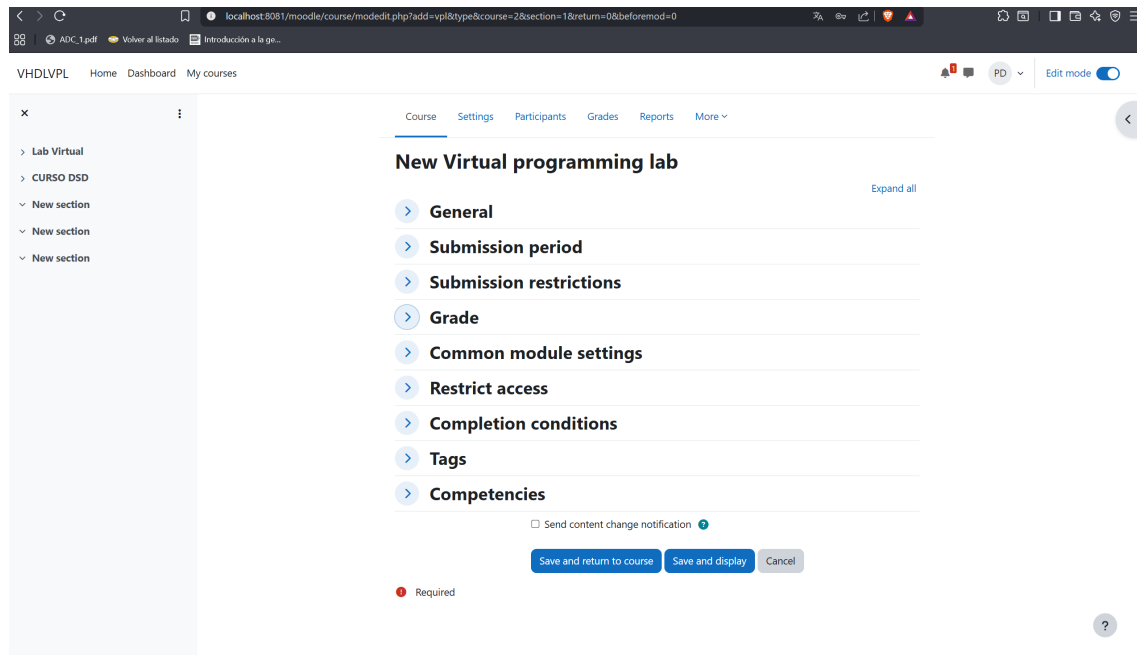


Figura 8.6: Guardar cambios en la actividad VPL

Una vez guardadas las configuraciones establecidas, la actividad debería verse como en la siguiente imagen:

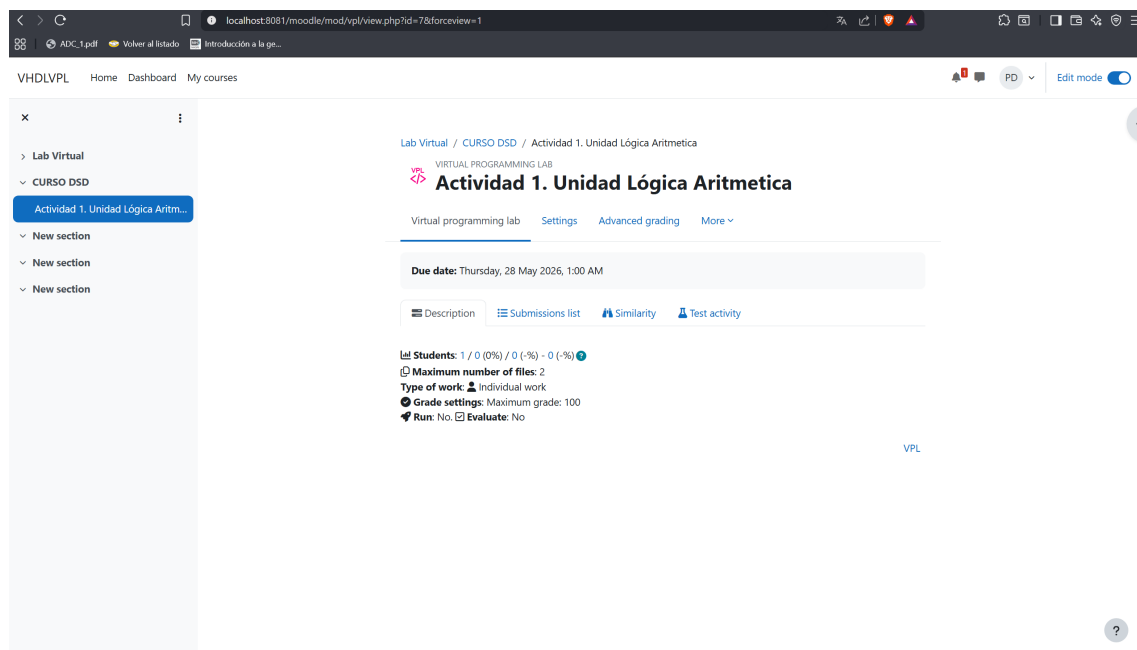


Figura 8.7: Visión general de la actividad VPL

Acciones Permitidas

En esta sección se describen como configurar las acciones que ofrece el entorno de VPL, esto respecto a la ejecución, depuración y evaluación de los diseños VHDL entregados por los alumnos. Es importante configurar esta sección una vez creada la actividad, ya que si visualizamos la imagen 8.7, observamos que por se visualiza la acción RUN habilitada y Evaluate deshabilitada, por lo que hay que configurar las acciones para permitir o no al alumno usarlas.

9.1 Tipos de acciones en VPL

En el laboratorio virtual de VPL, existen 5 tipos principales de acciones que se pueden configurar para los alumnos:

- **Run (Ejecutar):** Permite a los alumnos ejecutar su código VHDL para verificar su funcionamiento básico. Esta acción generalmente compila y simula el diseño, mostrando los resultados de la simulación.
- **Debug (Depurar):** Permite a los alumnos depurar su código VHDL, lo que puede incluir la visualización de formas de onda, inspección de señales y análisis detallado del comportamiento del diseño durante la simulación.
- **Evaluate (Evaluar):** Permite a los alumnos enviar su diseño para evaluación automática, donde se ejecutan casos de prueba predefinidos para verificar la corrección del diseño y asignar una calificación basada en los resultados.
- **Remote Lab (Laboratorio Remoto):** Permite conectarse al laboratorio remoto de FPGA'S.
- **Testbench (Generar Testbench):** Permite a los alumnos generar automáticamente un testbench básico para su diseño VHDL, lo que facilita la simulación y verificación de su código.

9.2 Configurar acciones disponibles para el alumno

Para habilitar o deshabilitar estas acciones para los alumnos, se deben seguir los siguientes pasos:

Paso 1

Ingresa a la actividad VPL creada y hacer clic en el botón **MAS / More** para que se desplieguen mas opciones del VPL.

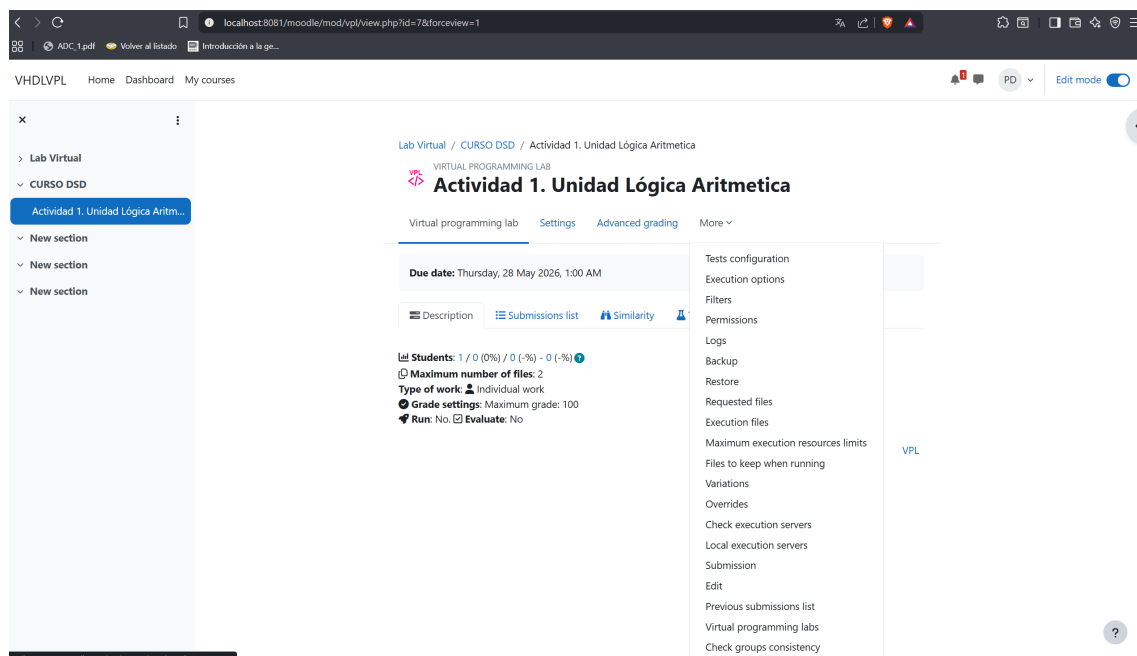
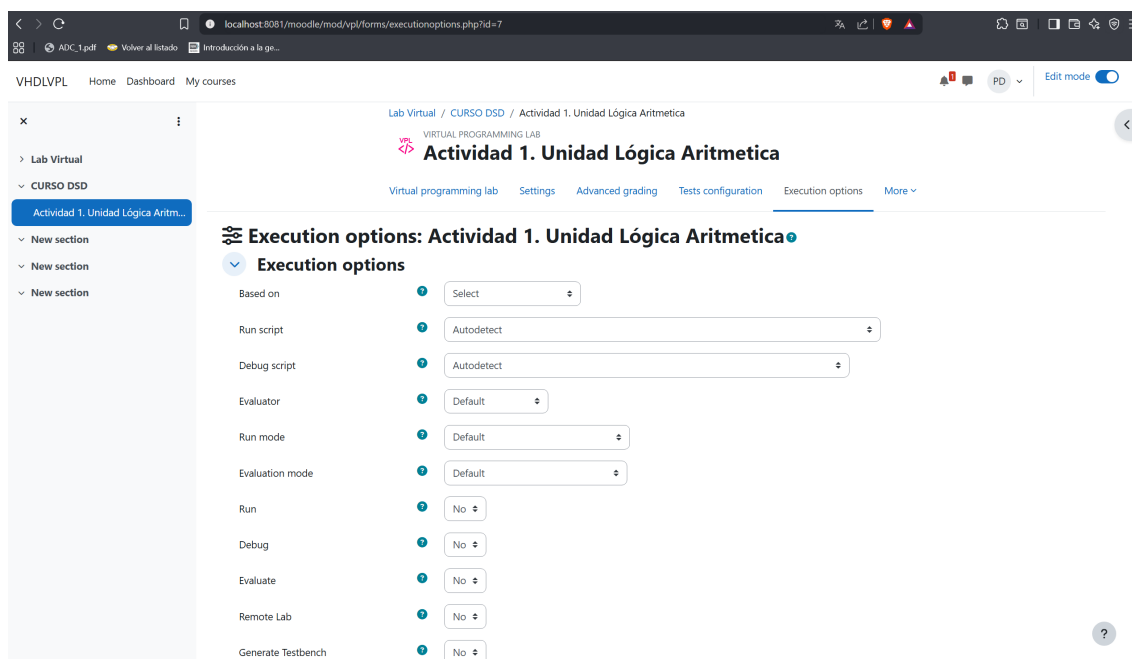


Figura 9.1: Mas configuraciones de la actividad VPL

Paso 2

En la sección de **Acciones de ejecución**, seleccionar las acciones que se desean habilitar para los alumnos.

9.2. CONFIGURAR ACCIONES DISPONIBLES PARA EL ALUMNO

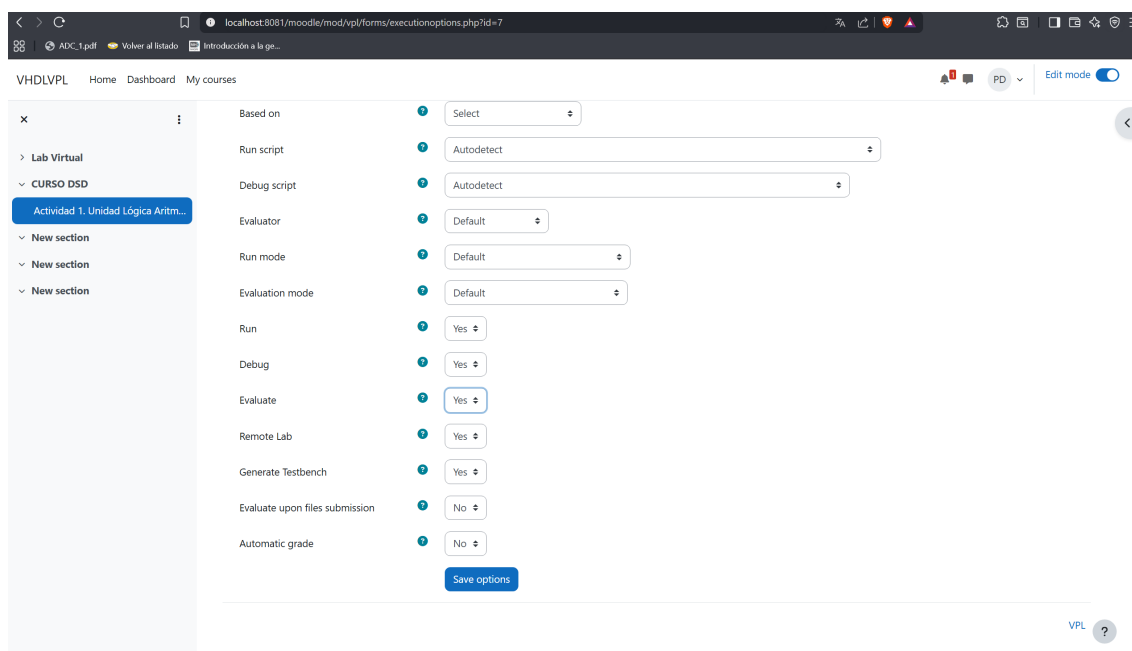


Execution options: Actividad 1. Unidad Lógica Aritmetica	
Based on	Select
Run script	Autodetect
Debug script	Autodetect
Evaluator	Default
Run mode	Default
Evaluation mode	Default
Run	No
Debug	No
Evaluate	No
Remote Lab	No
Generate Testbench	No

Figura 9.2: Opciones de ejecución en la actividad VPL

Paso 3

Dentro de esta sección lo mas importante es habilitar o deshabilitar las opciones de **Run** , **Debug** , **Remote Lab** , **Generate Testbench** y **Evaluate**, ya que estas son las acciones principales para que el alumno pueda ejecutar.



Execution options: Actividad 1. Unidad Lógica Aritmetica	
Based on	Select
Run script	Autodetect
Debug script	Autodetect
Evaluator	Default
Run mode	Default
Evaluation mode	Default
Run	Yes
Debug	Yes
Evaluate	Yes
Remote Lab	Yes
Generate Testbench	Yes
Evaluate upon files submission	No
Automatic grade	No

Save options

Figura 9.3: Permitir opciones de ejecución en la actividad VPL

Configuración de Casos de Prueba

El archivo `vpl_evaluate.cases` contiene los casos de prueba que el evaluador `vpl_evaluate.sh` ejecuta al calificar un diseño VHDL. El evaluador compila el código del alumno, detecta la entidad top, extrae sus puertos, genera un testbench VHDL automáticamente a partir de los casos, lo simula con GHDL y calcula la nota final.

10.1 Archivo `vpl_evaluate.cases`

El archivo tiene dos bloques diferenciados:

1. **Configuración global** — parámetros que aplican a todos los casos (reloj, reset, tiempo de simulación). Aparecen antes del primer `case`.
2. **Definición de casos** — uno o más bloques `case`, cada uno con sus secciones `input:` y `output:`.

Las líneas que comienzan con `#` son comentarios y se ignoran. Los nombres de parámetro son insensibles a mayúsculas y minúsculas.

10.1.1 Sintaxis general de los casos de prueba

Configuración global

Los parámetros globales se escriben como `parámetro = valor` y se colocan al inicio del archivo, antes de cualquier `case`. La Tabla 10.1 lista los parámetros disponibles.

Cuadro 10.1: Parámetros de configuración global

Parámetro	Ejemplo	Descripción
<code>stop_time_all</code>	<code>1 ms</code>	Tiempo máximo de simulación global.

Parámetro	Ejemplo	Descripción
clock	CLK	Nombre del puerto de reloj. Si se omite, el evaluador lo detecta automáticamente buscando puertos llamados <code>clk</code> , <code>clock</code> o <code>ck</code> .
clock_period	10 ns	Período completo del reloj.
reset	RESET	Nombre del puerto de reset. Si se omite se busca entre <code>reset</code> , <code>rst</code> , <code>clr</code> y variantes con sufijo <code>_n</code> .
reset_active	1	Nivel activo del reset: 1 = activo alto, 0 = activo bajo.
reset_type	async	Tipo de reset: <code>async</code> o <code>sync</code> .
reset_cycles	2	Ciclos de reloj durante los que se mantiene el reset al inicio de la simulación.

Definición de un caso

Cada caso comienza con la línea `case = descripción` y contiene los parámetros opcionales listados en la Tabla 10.2, seguidos de las secciones de señales.

Cuadro 10.2: Parámetros por caso de prueba

Parámetro	Ejemplo	Descripción
case	Prueba AND	Nombre del caso; aparece en los reportes de fallo.
stop_time	50 ns	Tiempo de simulación específico para este caso. Sobreescribe <code>stop_time_all</code> solo para este caso.
grade reduction	25 %	Penalización si el caso falla. Se admite porcentaje (%) o valor absoluto. Si se omite, la penalización se reparte equitativamente entre todos los casos.

Secciones de señales

Dentro de cada caso se declaran dos secciones:

- **input:** — valores que se aplican a los puertos de entrada.
- **output:** — valores esperados en los puertos de salida.

Cada señal se especifica en una línea indentada con el formato:

NOMBRE_SEÑAL = valor

El valor puede ser:

- **Escalar** — un único vector de bits: $A = 1010$.
- **Array** — varios valores para múltiples ciclos de reloj: $X = [0, 1, 1, 0]$.
- **Comodín** — guion bajo `_` dentro de un array de salida indica que ese ciclo no se verifica: $Z = [_, _, 1, 0]$.

Si alguna señal de entrada o salida usa sintaxis de array, el evaluador activa el **modo secuencial**: genera un testbench que aplica un valor por ciclo de reloj y verifica la salida al final de cada ciclo. Sin arrays y sin reloj detectado, opera en **modo combinatorio**: aplica las entradas, espera `stop_time` y verifica las salidas una sola vez.

10.1.2 Sintaxis específica para evaluar código VHDL

Circuitos combinatorios

Para circuitos sin reloj se omite la configuración global de clock/reset. El evaluador aplica los valores de `input:`, espera `stop_time` y compara las salidas declaradas en `output:`.

```

1  # Tiempo de espera por defecto para todos los casos
2  stop_time_all = 50 ns
3
4  case = Suma basica sin acarreo
5  stop_time = 20 ns
6  grade reduction = 50%
7  input:
8      A   = 0011
9      B   = 0101
10     Co  = 0
11  output:
12     S   = 1000
13     C4  = 0
14
15  case = Suma con acarreo de entrada
16  stop_time = 20 ns
17  grade reduction = 50%
18  input:
19     A   = 1111

```

```

20     B  = 0001
21     Co = 1
22 output:
23     S  = 0001
24     C4 = 1

```

Listing 10.1: Caso combinatorio — sumador de 4 bits con acarreo*Circuitos secuenciales*

Para circuitos con reloj se declara la configuración global antes del primer `case`. Los valores de entrada y salida se escriben como arrays [`v0`, `v1`, ...], donde cada elemento corresponde a un ciclo de reloj. El evaluador genera el reloj, aplica el reset durante `reset_cycles` ciclos y luego recorre el array ciclo a ciclo.

```

1  clock          = CLK
2  clock_period   = 10 ns
3  reset          = RESET
4  reset_active   = 0          # activo bajo
5  reset_type     = async
6  reset_cycles   = 2
7  stop_time_all  = 1 ms
8
9  # Secuencia X=[0,0,1,1] desde estado inicial C
10 # Transiciones: C->B->A->B->C
11 case = Recorrido C-B-A-B-C
12 grade reduction = 60%
13 input:
14     X = [0, 0, 1, 1]
15 output:
16     Z1 = [1, 0, 1, 1]
17     Z2 = [0, 1, 1, 0]
18
19 case = Una transicion C a B
20 grade reduction = 40%
21 input:
22     X = [0]
23 output:
24     Z1 = [1]
25     Z2 = [0]

```

Listing 10.2: Caso secuencial — FSM Mealy**Longitud de arrays**

Los arrays de `input:` y `output:` de un mismo caso deben tener la misma longitud. El evaluador no rellena valores faltantes.

Uso del comodín _

El comodín `_` en un array de salida indica que ese ciclo no se verifica. Es útil cuando la salida del circuito no está definida durante los primeros ciclos (por ejemplo, mientras se llena un pipeline o un registro de desplazamiento).

```

1 clock          = CLK
2 clock_period   = 10 ns
3 reset          = CLR
4 reset_active   = 1
5 reset_type     = async
6 reset_cycles   = 2
7 stop_time_all  = 1 ms
8
9 # Ciclo 1: carga L=1 -> Q=1000000000000
10 # Ciclo 2: desplaza SHE=1 -> Q=0100000000000
11 # Ciclo 3: desplaza SHE=1 -> Q=0010000000000
12 case = Cargar y desplazar
13 grade reduction = 35%
14 input:
15     L    = [1, 0, 0]
16     SHE  = [0, 1, 1]
17     ES   = [0, 0, 0]
18     D    = 1000000000000
19 output:
20     LED  = [_, 0100010000000000, 0100001000000000]
```

Listing 10.3: Comodín en salidas de registro de desplazamiento

Test de Evaluación por Parte del Profesor



Glosario de Términos

B

Referencia Rápida de Comandos

Recursos y Referencias
